Operating Systems

Module 4: Concurrency

Process Synchronization

Dr. Suresh P

Module 3: Process Synchronization

- Background
- The Critical-Section Problem
- Peterson's Solution
- Synchronization Hardware
- Semaphores
- Classic Problems of Synchronization
- Monitors
- Synchronization Examples
- Atomic Transactions



Objectives

- To introduce the critical-section problem, whose solutions can be used to ensure the consistency of shared data
- To present both software and hardware solutions of the critical-section problem
- To introduce the concept of an atomic transaction and describe mechanisms to ensure atomicity



Interprocess Communication

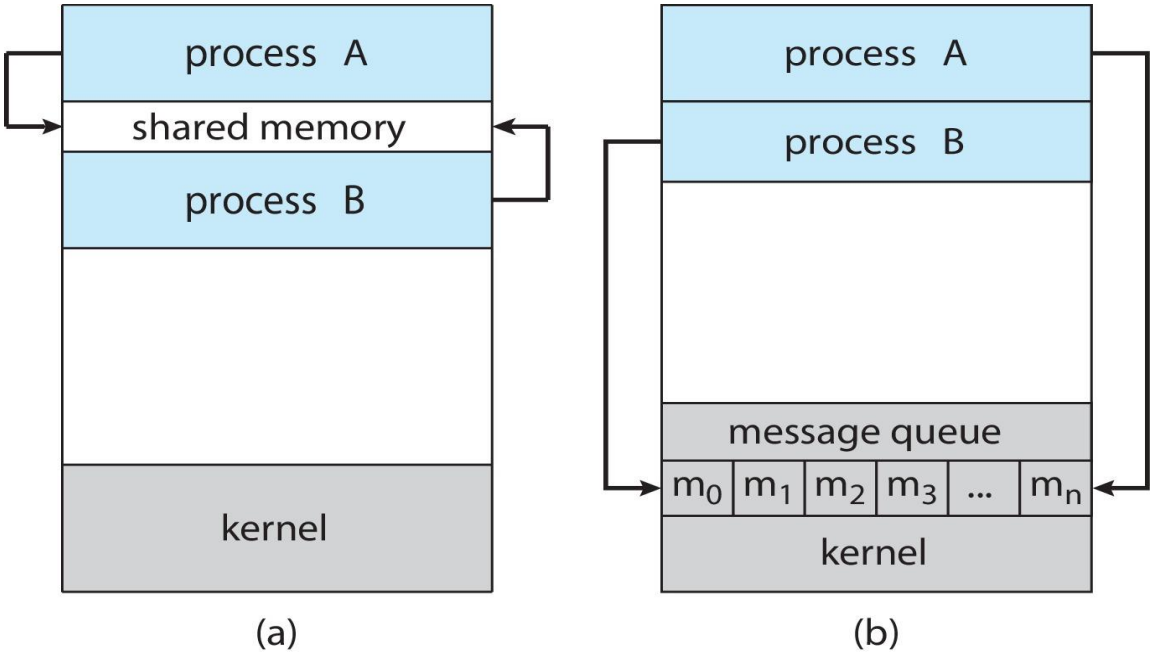
- Processes within a system may be ***independent*** or ***cooperating***
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- Cooperating processes need **interprocess communication** (**IPC**)
- Two models of IPC
  - **Shared memory**
  - **Message passing**



Communications Models

(a) Shared memory.

(b) Message passing.



Paradigm for cooperating processes

- Paradigm for cooperating processes:
  - *producer* process produces information that is consumed by a *consumer* process
- Two variations:
  - **unbounded-buffer** places no practical limit on the size of the buffer:
    - Producer never waits
    - Consumer waits if there is no buffer to consume
  - **bounded-buffer** assumes that there is a fixed buffer size
    - Producer must wait if all buffers are full
    - Consumer waits if there is no buffer to consume



IPC – Shared Memory

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system.
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.



Bounded-Buffer – Shared-Memory Solution

- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
  . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- Solution is correct, but can only use **`BUFFER_SIZE-1`** elements



Producer Process – Shared Memory

```
item next_produced;

while (true) {
   /* produce an item in next produced */
   while (((in + 1) % BUFFER_SIZE) == out) ; /* do nothing */
   buffer[in] = next_produced;
   in = (in + 1) % BUFFER_SIZE;
}
```




Consumer Process – Shared Memory

```
item next_consumed;

while (true) {
        while (in == out); /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;

        /* consume the item in next consumed */
}
```




What about Filling all the Buffers?

- Suppose that we wanted to provide a solution to the consumer-producer problem that fills **all** the buffers.
- We can do so by having an integer **`counter`** that keeps track of the number of full buffers.
- Initially, **`counter`** is set to 0.
- The integer **`counter`** is incremented by the producer after it produces a new buffer.
- The integer **`counter`** is and is decremented by the consumer after it consumes a buffer.



Producer

```
while (true) {
        /* produce an item in next
produced */

        while (counter ==
BUFFER_SIZE); /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
        counter++;
}
```

Consumer

```
while (true) {
        while (counter == 0)
                ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
      counter--;
        /* consume the item in next consumed */
}
```



Race Condition

- **`counter++`** could be implemented as

```
register1 = counter
register1 = register1 + 1
counter = register1
```

- **`counter--`** could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- Consider this execution interleaving with "count = 5" initially:

S0: producer execute `register1 = counter` {register1 = 5}
S1: producer execute `register1 = register1 + 1` {register1 = 6}
S2: consumer execute `register2 = counter` {register2 = 5}
S3: consumer execute `register2 = register2 - 1` {register2 = 4}
S4: producer execute `counter = register1` {counter = 6 }
S5: consumer execute `counter = register2` {counter = 4}



IPC – Message Passing

- Processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
  - **`send`**(*message*)
  - **`receive`**(*message*)
- The *message* size is either fixed or variable



Message Passing (Cont.)

- If processes *P* and *Q* wish to communicate, they need to:
  - Establish a ***communication link*** between them
  - Exchange messages via send/receive
- Implementation issues:
  - How are links established?
  - Can a link be associated with more than two processes?
  - How many links can there be between every pair of communicating processes?
  - What is the capacity of a link?
  - Is the size of a message that the link can accommodate fixed or variable?
  - Is a link unidirectional or bi-directional?



Implementation of Communication Link

- Logical:
  - Direct or indirect
  - Synchronous or asynchronous
  - Automatic or explicit buffering



Direct Communication

- Processes must name each other explicitly:
  - **`send`** (*P, message*) – send a message to process P
  - **`receive`**(*Q, message*) – receive a message from process Q
- Properties of communication link
  - Links are established automatically
  - A link is associated with exactly one pair of communicating processes
  - Between each pair there exists exactly one link
  - The link may be unidirectional, but is usually bi-directional



Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
  - Each mailbox has a unique id
  - Processes can communicate only if they share a mailbox
- Properties of communication link
  - Link established only if processes share a common mailbox
  - A link may be associated with many processes
  - Each pair of processes may share several communication links
  - Link may be unidirectional or bi-directional



Indirect Communication (Cont.)

- Operations
  - Create a new mailbox (port)
  - Send and receive messages through mailbox
  - Delete a mailbox
- Primitives are defined as:
  - **`send`**(*A, message*) – send a message to mailbox A
  - **`receive`**(*A, message*) – receive a message from mailbox A



Indirect Communication (Cont.)

- Mailbox sharing
  - P_1, P_2, and P_3 share mailbox A
  - P_1, sends; P_2 and P_3 receive
  - Who gets the message?
- Solutions
  - Allow a link to be associated with at most two processes
  - Allow only one process at a time to execute a receive operation
  - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.



Synchronization

Message passing may be either blocking or non-blocking

- **Blocking** is considered **synchronous**
  - **Blocking send** -- the sender is blocked until the message is received
  - **Blocking receive** -- the receiver is blocked until a message is available
- **Non-blocking** is considered **asynchronous**
  - **Non-blocking send** -- the sender sends the message and continue
  - **Non-blocking receive** -- the receiver receives:
    - A valid message, or
    - Null message
- Different combinations possible
  - If both send and receive are blocking, we have a **rendezvous**



Producer-Consumer: Message Passing

- Producer

```
    message next_produced;
    while (true) {
        /* produce an item in
next_produced */

     send(next_produced);
    }
```

- Consumer

```
     message next_consumed;
    while (true) {
         receive(next_consumed)

        /* consume the item in
next_consumed */
   }
```



Buffering

- Queue of messages attached to the link.
- Implemented in one of three ways
  1. Zero capacity – no messages are queued on a link.
     Sender must wait for receiver (rendezvous)
  2. Bounded capacity – finite length of n messages
     Sender must wait if link full
  3. Unbounded capacity – infinite length
     Sender never waits



Concurrent Execution

- Concurrent execution has to give the same results as serial execution.
- Concurrent execution with shared data leads us to speak about synchronization.
- To get data consistency we should have mechanism to avoid data inconsistency problem.
- Synchronization as embedded system topic we have to speak about producer consumer problem



- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes

- Suppose that we wanted to provide a solution to the consumer-producer problem that fills **all** the buffers. We can do so by having an integer **count** that keeps track of the number of full buffers. Initially, count is set to 0.

- It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.



Producer Consumer Problem

- The producer-consumer problem illustrates the need for synchronization in systems where many processes share a resource.
- In the problem, two processes share a fixed-size buffer. One process produces information and puts it in the buffer, while the other process consumes information from the buffer.
- These processes do not take turns accessing the buffer, they both work concurrently.
- It is also called bounded buffer problem



Producer

```
while (true) {
        /*  produce an item and put in nextProduced  */
    while (counter == BUFFER_SIZE)
            ; // do nothing
        buffer [in] = nextProduced;
        in = (in + 1) % BUFFER_SIZE;
        counter++;
}
```



CONSUMER

```
while (true)  {
        while (counter == 0)
                ; // do nothing
                nextConsumed =  buffer[out];
                 out = (out + 1) % BUFFER_SIZE;
               counter--;
                      /*  consume the item in nextConsumed  */
}
```



Race Condition

- counter++ could be implemented as

```
register1 = counter
register1 = register1 + 1
counter = register1
```

- counter-- could be implemented as

```
register2 = counter
register2 = register2 - 1
count = register2
```

- Consider this execution interleaving with "count = 5" initially:

S0: producer execute register1 = counter {register1 = 5}
S1: producer execute register1 = register1 + 1 {register1 = 6}
S2: consumer execute register2 = counter {register2 = 5}
S3: consumer execute register2 = register2 - 1 {register2 = 4}
S4: producer execute counter = register1 {count = 6 }
S5: consumer execute counter = register2 {count = 4}



- Now we have arrived at the incorrect state "counter == 4", indicating that four buffers are full, If we reversed the order of the statements at S4 and S5, we would arrive at the incorrect state "counter —— 6".

- We would arrive at this incorrect state because we allowed both processes to manipulate the variable counter concurrently.

- A situation like this, where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a **race condition.**

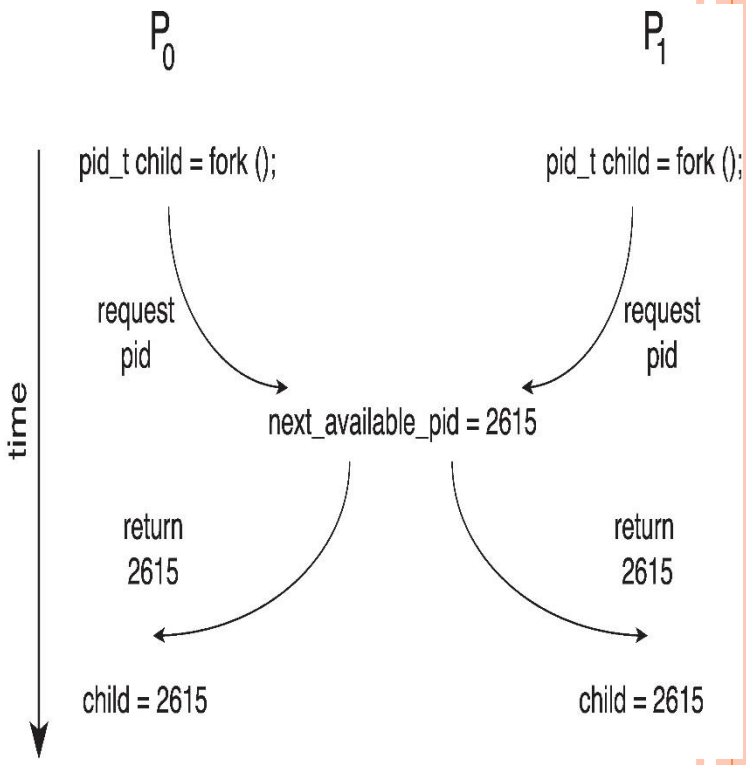
- To avoid the race condition, we need to ensure that only one process at a time can be manipulating the variable counter. To make such a guarantee, we require that the processes be synchronized in some way.



Synchronization

Race Condition

- Processes P_0 and P_1 are creating child processes using the `fork()` system call
- Race condition on kernel variable `next_available_pid` which represents the next available process identifier (pid)
- Unless there is a mechanism to prevent P_0 and P_1 from accessing the variable `next_available_pid` the same pid could be assigned to two different processes!



Critical Section Problem

- Consider system of ***n*** processes $\{p_0, p_1, \ldots p_{n-1}\}$
- Each process has **critical section** segment of code
  - Process may be changing common variables, updating table, writing file, etc.
  - When one process in critical section, no other may be in its critical section
- ***Critical section problem*** is to design protocol to solve this
- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**

Critical Section

- General structure of process P_i

```
while (true) {
        entry section
            critical section
        exit section
            remainder section
}
```

Critical-Section Problem (Cont.)

Requirements for solution to critical-section problem

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the process that will enter the critical section next cannot be postponed indefinitely
3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
   - Assume that each process executes at a nonzero speed
   - No assumption concerning **relative speed** of the ***n*** processes



Peterson's Solution

- Two process solution
- Assume that the **`load`** and **`store`** machine-language instructions are atomic; that is, cannot be interrupted
- The two processes share two variables:
  - **`int turn;`**
  - **`boolean flag[2]`**
- The variable **`turn`** indicates whose turn it is to enter the critical section
- The **`flag`** array is used to indicate if a process is ready to enter the critical section.
  - **`flag[i] = true`** implies that process $\mathbf{P_i}$ is ready!

Algorithm for Process P_I

```
while (true){

        flag[i] = true;
        turn = j;
        while (flag[j] && turn = = j)
                ;

            /* critical section */

        flag[i] = false;

        /* remainder section */

}
```

Process 0	Process 1
flag[0] = true;	flag[1] = true;
turn = 1;	turn = 0;
while (flag[1] == true && turn == 1) Busy wait	while (flag[0] == true && turn == 0) Busy wait
// Critical Section starts // access shared resources here // Critical Section ends	*// Critical Section starts // access shared resources here // Critical Section ends*
flag[0] = false;	flag[1] = false;

Correctness of Peterson's Solution

- Provable that the three CS requirement are met:
  1. Mutual exclusion is preserved

     P_i enters CS only if:

     either **`flag[j] = false`** or **`turn = i`**
  2. Progress requirement is satisfied
  3. Bounded-waiting requirement is met



Bakery Algorithm

- Bakery Algorithm is a critical section solution for **N** processes.
- The algorithm preserves the first come first serve property.
- Each process is assigned a number (a ticket) in a lexicographical order.
- Before entering the critical section, a process receives a ticket number, and the process with the smallest ticket number enters the critical section.
- If two processes receive the same ticket number, the process with the lower process ID is given priority.



Pseudocode- bakery algorithm

```
repeat
    choosing[i] := true;
    number[i] := max(number[0], number[1], ..., number[n - 1])+1;
    choosing[i] := false;
    for j := 0 to n - 1
        do begin
            while choosing[j] do no-op;
            while number[j] != 0
                and (number[j], j) < (number[i], i) do no-op;
        end;

        critical section

    number[i] := 0;

        remainder section

until false;
```

Synchronization Hardware

- Many systems provide hardware support for implementing the critical section code.
- Uniprocessors – could disable interrupts
  - Currently running code would execute without preemption
  - Generally too inefficient on multiprocessor systems
    - Operating systems using this not broadly scalable
- We will look at three forms of hardware support:

1. Hardware instructions
2. Atomic variables



Hardware Instructions

- Special hardware instructions that allow us to either *test-and-modify* the content of a word, or to *swap* the contents of two words atomically (uninterruptedly.)
  - **Test-and-Set** instruction
  - **Compare-and-Swap** instruction



The test_and_set Instruction

- Definition

```
boolean test_and_set (boolean *target)
 {
      boolean rv = *target;
      *target = true;
      return rv:
 }
```

- Properties
  - Executed atomically
  - Returns the original value of passed parameter
  - Set the new value of passed parameter to **`true`**

Solution Using test_and_set()

- Shared boolean variable **lock**, initialized to **false**
- Solution:

```
do {
    while (test_and_set(&lock))
      ; /* do nothing */

        /* critical section */

   lock = false;
        /* remainder section */
} while (true);
```

- Does it solve the critical-section problem?

The compare_and_swap Instruction

- Definition

```
int compare_and_swap(int *value, int expected, int new_value)
 {
  int temp = *value;
   if (*value == expected)
      *value = new_value;
    return temp;
 }
```

- Properties
  - Executed atomically
  - Returns the original value of passed parameter **`value`**
  - Set the variable **`value`** the value of the passed parameter **`new_value`** but only if **`*value == expected`** is true. That is, the swap takes place only under this condition.



Solution using compare_and_swap

- Shared integer **`lock`** initialized to 0;
- Solution:

```
while (true){
        while (compare_and_swap(&lock, 0, 1) != 0)
        ; /* do nothing */

        /* critical section */

        lock = 0;

    /* remainder section */
 }
```

- Does it solve the critical-section problem?

Bounded-waiting with compare-and-swap

```
while (true) {
  waiting[i] = true;
  key = 1;
  while (waiting[i] && key == 1)
    key = compare_and_swap(&lock,0,1);
  waiting[i] = false;
  /* critical section */


  j = (i + 1) % n;
   while ((j != i) && !waiting[j])
     j = (j + 1) % n;
   if (j == i)
     lock = 0;
   else
     waiting[j] = false;
   /* remainder section */
 }
```

Mutex Locks

- Previous solutions are complicated and generally inaccessible to application programmers
- OS designers build software tools to solve critical section problem
- Simplest is mutex lock
  - Boolean variable indicating if lock is available or not
- Protect a critical section by
  - First **`acquire()`** a lock
  - Then **`release()`** the lock
- Calls to **`acquire()`** and **`release()`** must be **atomic**
  - Usually implemented via hardware atomic instructions such as compare-and-swap.
- But this solution requires **busy waiting**
  - This lock therefore called a **spinlock**



Solution to CS Problem Using Mutex Locks

```
while (true) {
        acquire lock

            critical section

        release lock

remainder section
}
```

SEMAPHORE

- A semaphore is hardware or a software variable whose value indicates the status of a common resource. Its purpose is to lock the resource being used.
- A process which needs the resource will check the semaphore for determining the status of the resource followed by the decision for proceeding.
- In multitasking operating systems, the activities are synchronized by using the semaphore techniques.
- Example, say we have four rooms with identical locks and keys. The semaphore count - the count of keys - is set to 4 at beginning (all four rooms are free), then the count value is decremented as people are coming in. If all rooms are full, ie. there are no free keys left, the semaphore count is 0. Now, when eq. one person leaves the rooms, semaphore is increased to 1 (one free key), and given to the next person in the queue.
- "A semaphore restricts the number of simultaneous users of a shared resource up to a maximum number. Threads can request access to the resource (decrementing the semaphore), and can signal that they have finished using the resource (incrementing the semaphore)."



- Synchronization tool that does not require busy waiting
- Semaphore *S* – integer variable
- Two standard operations modify S: wait() and signal()
  - Originally called P() and V()
- Less complicated

- Can only be accessed via two indivisible (atomic) operations
  - wait (S) {
    while (S <= 0); // no-op
    S--;
    }
  - signal (S) {
    S++;
    }



Semaphore as General Synchronization Tool

- **Counting** semaphore – integer value can range over an unrestricted domain
- **Binary** semaphore – integer value can range only between 0 and 1; can be simpler to implement
  - Also known as **mutex locks**
- Can implement a counting semaphore S as a binary semaphore
- Provides mutual exclusion

```
Semaphore mutex;    //  initialized to 1
do {
    wait (mutex);
        // Critical Section
     signal (mutex);
            // remainder section
} while (TRUE);
```



Semaphore Implementation

- Must guarantee that no two processes can execute wait () and signal () on the same semaphore at the same time

- Thus, implementation becomes the critical section problem where the wait and signal code are placed in the critical section
  - Could now have **busy waiting** in critical section implementation
    - But implementation code is short
    - Little busy waiting if critical section rarely occupied

- Note that applications may spend lots of time in critical sections and therefore this is not a good solution



Semaphore Implementation with no Busy waiting

- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
  - value (of type integer)
  - pointer to next record in the list
- Two operations:
  - **block** – place the process invoking the operation on the appropriate waiting queue
  - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue



Semaphore Implementation with no Busy waiting (Cont.)

- Implementation of wait:

```
wait(semaphore *S) {
        S->value--;
        if (S->value < 0) {
                add this process to S->list;
                block();
        }
}
```

- Implementation of signal:

```
signal(semaphore *S) {
        S->value++;
        if (S->value <= 0) {
                remove a process P from S->list;
                wakeup(P);
        }
}
```



Deadlock and Starvation

- **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let S and Q be two semaphores initialized to 1

P_0	P_1
wait (S);	wait (Q);
wait (Q);	wait (S);
.	.
.	.
.	.
signal (S);	signal (Q);
signal (Q);	signal (S);

- **Starvation** – indefinite blocking
  - A process may never be removed from the semaphore queue in which it is suspended
- **Priority Inversion** – Scheduling problem when lower-priority process holds a lock needed by higher-priority process
  - Solved via **priority-inheritance protocol**



Classical Problems of Synchronization

- Classical problems used to test newly-proposed synchronization schemes
  - Bounded-Buffer Problem
  - Readers and Writers Problem
  - Dining-Philosophers Problem



Bounded-Buffer Problem

- *N* buffers, each can hold one item
- Semaphore mutex initialized to the value 1
- Semaphore full initialized to the value 0
- Semaphore empty initialized to the value N



Bounded Buffer Problem (Cont.)

- The structure of the producer process

```
do {
        // produce an item in nextp

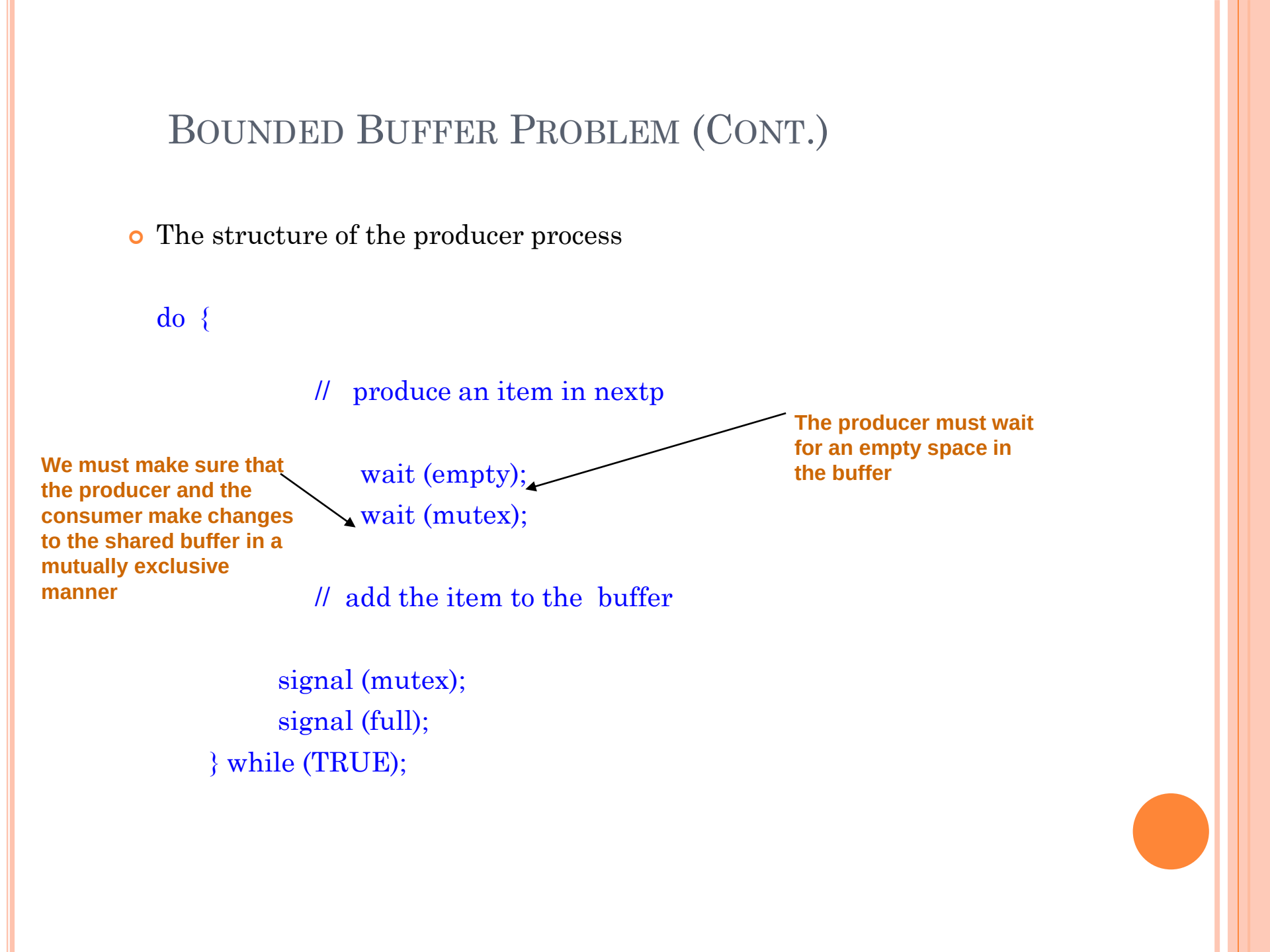
        wait (empty);
        wait (mutex);

        // add the item to the buffer

        signal (mutex);
        signal (full);
} while (TRUE);
```

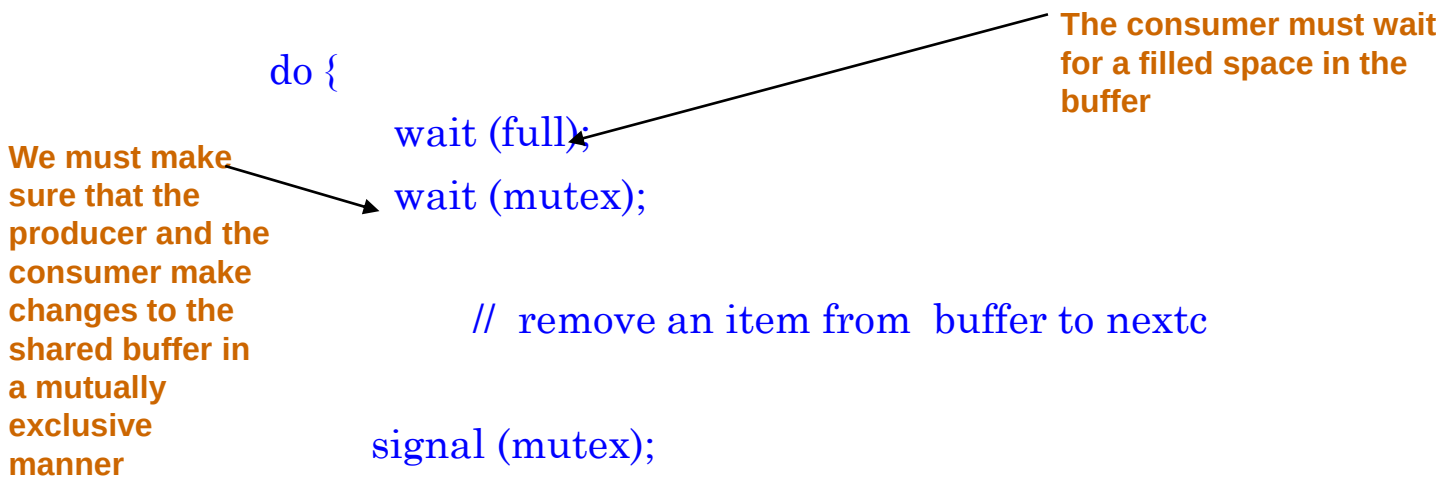
The producer must wait for an empty space in the buffer

We must make sure that the producer and the consumer make changes to the shared buffer in a mutually exclusive manner

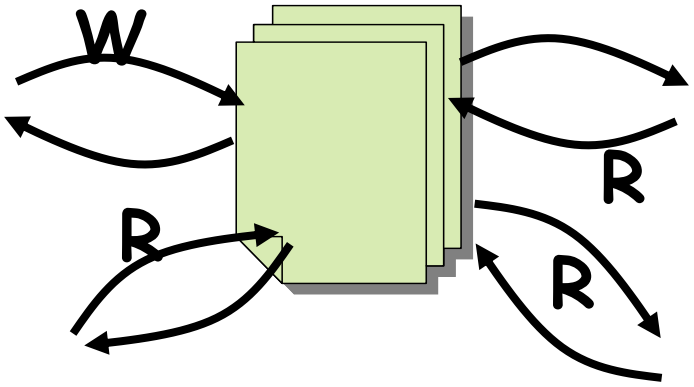


Bounded Buffer Problem (Cont.)

- The structure of the consumer process



Readers/Writers Problem



- Motivation: Consider a shared database
  - Two classes of users:
    - Readers – never modify database
    - Writers – read and modify database
  - Is using a single lock on the whole database sufficient?
    - Like to have many readers at the same time
    - Only one writer at a time



Readers/Writers Problem

- A database is to be shared among several concurrent processes. Some of these processes may want only to read the database, whereas others may want to update the database
- We distinguish between these two types of processes by referring to the former as readers and to the latter as writers
- Obviously, if two readers access the shared data simultaneously, nothing bad will happen
- However, if a writer and some other process (either a reader or a writer) access the database simultaneously, chaos may ensue



Readers/Writers Problem

- To ensure that these difficulties do not arise, we require that the writers have exclusive access to the shared database
- There are several variations of this problem, all involving priorities
  - The first and simplest one, referred to as the first readers/writers problem, requires that no reader will be kept waiting unless a writer has already obtained permission to use the shared object (i.e., no reader should wait for other readers to finish simply because a writer is waiting) NOTE: writers may starve
  - The second readers/writers problem requires that, once a writer is ready, that writer performs its write as soon as possible (i.e., if a writer is waiting, no new readers may start reading) NOTE: readers may starve



Readers-Writers Problem (Cont.)

- The structure of a writer process



A writer will wait if either another writer is currently writing or one or more readers are currently reading

```
do {
        wait (wrt) ;

            //    writing is performed

        signal (wrt) ;
} while (TRUE);
```



Readers-Writers Problem (Cont.)

- The structure of a reader process

```
do {
        wait (mutex) ;
        readcount ++ ;
        if (readcount == 1)
                wait (wrt) ;
        signal (mutex)

            // reading is performed

        wait (mutex) ;
        readcount  - - ;
        if (readcount  == 0)
                signal (wrt) ;
        signal (mutex) ;
} while (TRUE);
```

We must make sure that readers update the shared variable **readcount** in a mutually exclusive manner

A reader will wait only if a writer is currently writing. Note that if **readcount == 1**, no reader is currently reading and thus that is the only time that a reader has to make sure that no writer is currently writing (i.e., if **readcount > 1**, there is at least one reader reading and thus the new reader does not have to wait)



Dining-Philosophers Problem



- Philosophers spend their lives thinking and eating
- Don't interact with their neighbors, occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
  - Need both to eat, then release both when done
- In the case of 5 philosophers
  - Shared data
    - Bowl of rice (data set)
    - Semaphore chopstick [5] initialized to 1

Dining-Philosophers Problem Algorithm

- The structure of Philosopher *i*:

```
do {
        wait ( chopstick[i] );
         wait ( chopStick[ (i + 1) % 5] );

              //  eat

         signal ( chopstick[i] );
         signal (chopstick[ (i + 1) % 5] );

             //  think

} while (TRUE);
```

A philosopher must wait for his/her left and right chopsticks to be available before he/she can start eating

This solution guarantees that no two neighbors can be eating simultaneously (i.e., mutual exclusion)

- What is the problem with this algorithm?

  This solution could create a deadlock. How?



Problems with Semaphores

- Incorrect use of semaphore operations:
  - signal (mutex) …. wait (mutex)
  - wait (mutex) … wait (mutex)
  - Omitting of wait (mutex) or signal (mutex) (or both)
- Deadlock and starvation



Monitors

- A high-level abstraction that provides a convenient and effective mechanism for process synchronization
- Goal of OS is to share resources amongst many programs.
- Separate schedulers should be created for each class of resource.
- Each scheduler contains local data + procedures that programs may use to acquire and release resources. Such a collection of data + procedures is a monitor.

- *Abstract data type*, internal variables only accessible by code within the procedure
- Only one process may be active within the monitor at a time If more than one program attempts to enter at the same time, only one will succeed, and the remaining programs will remain on a queue.
-

- But not powerful enough to model some synchronization schemes



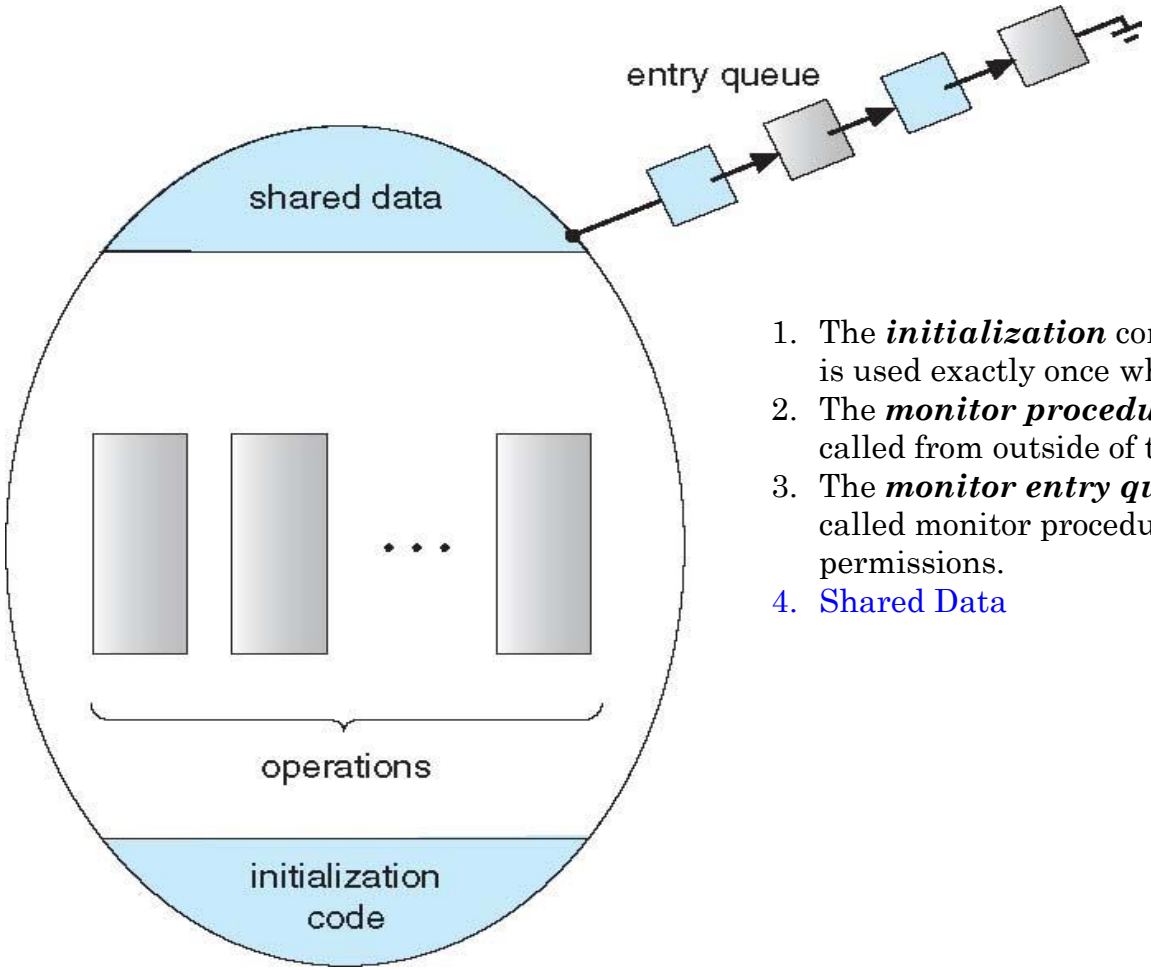
```
monitor monitor-name
{
  // shared variable declarations
  procedure P1 (…) { …. }

  procedure Pn (…) {……}

   Initialization code (…) { … }
  }
}
```



Schematic view of a Monitor



1. The ***initialization*** component contains the code that is used exactly once when the monitor is created
2. The ***monitor procedures*** are procedures that can be called from outside of the monitor.
3. The ***monitor entry queue*** contains all threads that called monitor procedures but have not been granted permissions.
4. Shared Data

Condition Variables

- condition x, y;

- Two operations on a condition variable:
  - x.wait () – a process that invokes the operation is suspended until x.signal ()

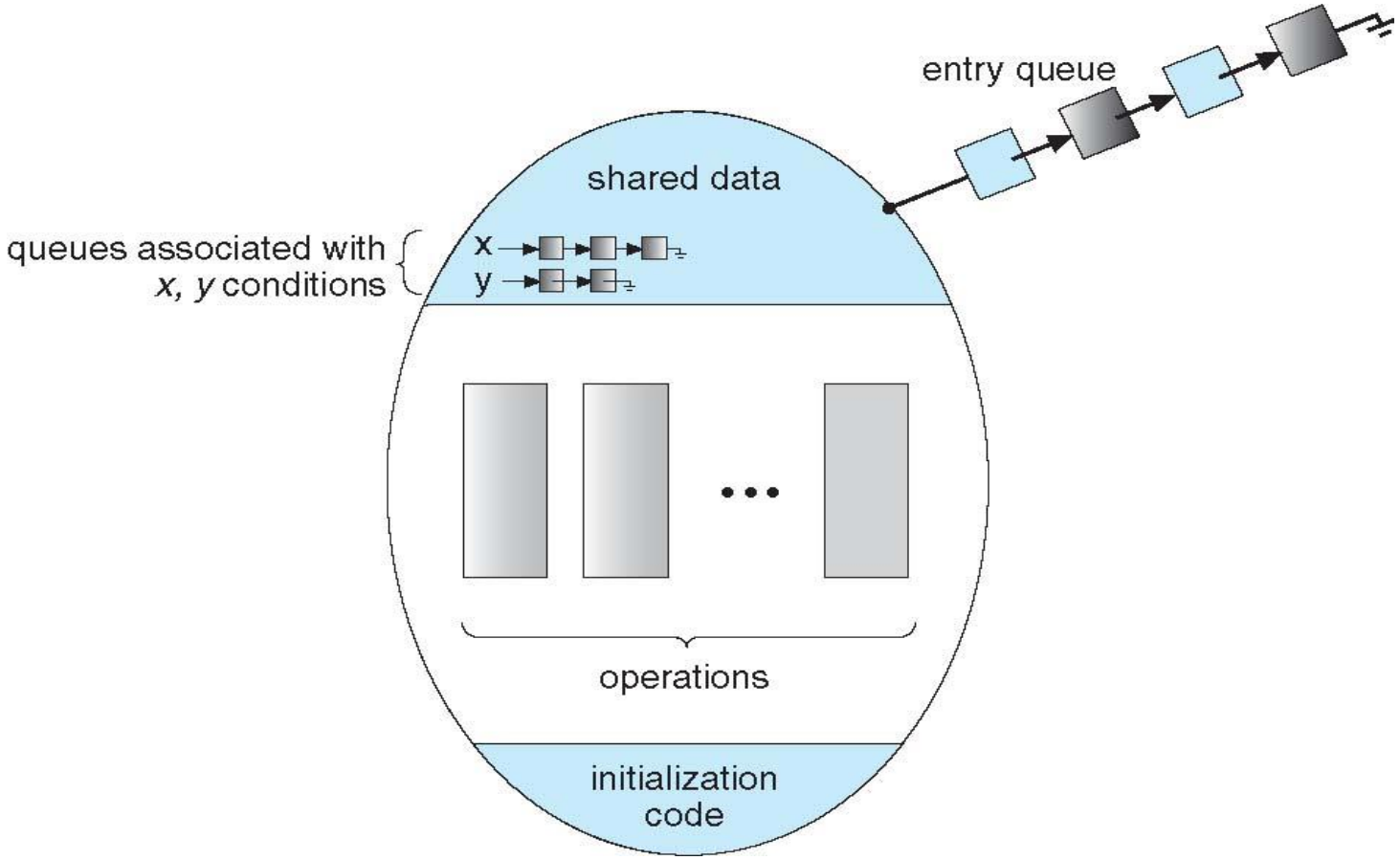
    If a procedure calls wait, the calling program will block until some other procedure calls signal.

  - x.signal () – resumes one of processes (if any) that invoked x.wait ()

    When a procedure calls signal, then the lock on the monitor is also released, and another program that previously called wait will run immediately.
    - If no x.wait () on the variable, then it has no effect on the variable.



Monitor with Condition Variables



Condition Variables Choices

- If process P invokes x.signal (), with Q in x.wait () state, what should happen next?
  - If Q is resumed, then P must wait
- Options include
  - **Signal and wait** – P waits until Q leaves monitor or waits for another condition
  - **Signal and continue** – Q waits until P leaves the monitor or waits for another condition
  - Both have pros and cons – language implementer can decide
  - Monitors implemented in Concurrent Pascal compromise
    - P executing signal immediately leaves the monitor, Q is resumed
  - Implemented in other languages including Mesa, C#, Java



Solution to Dining Philosophers

```
monitor DiningPhilosophers
  {
   enum { THINKING; HUNGRY, EATING) state [5] ;
   condition self [5];

   void pickup (int i) {
         state[i] = HUNGRY;
         test(i);
         if (state[i] != EATING) self [i].wait;
   }

      void putdown (int i) {
         state[i] = THINKING;
               // test left and right neighbors
          test((i + 4) % 5);
          test((i + 1) % 5);
       }
```



Solution to Dining Philosophers (Cont.)

```
    void test (int i) {
            if ( (state[(i + 4) % 5] != EATING) &&
            (state[i] == HUNGRY) &&
            (state[(i + 1) % 5] != EATING) ) {
                state[i] = EATING ;
                    self[i].signal () ;
             }
     }

       initialization_code() {
           for (int i = 0; i < 5; i++)
           state[i] = THINKING;
    }
}
```



Solution to Dining Philosophers (Cont.)

- Each philosopher *i* invokes the operations pickup() and putdown() in the following sequence:

```
DiningPhilosophers.pickup (i);

    EAT

DiningPhilosophers.putdown (i);
```

- No deadlock, but starvation is possible



Monitor Implementation Using Semaphores

- Variables

```
semaphore mutex;  // (initially  = 1)
semaphore next;     // (initially  = 0)
int next_count = 0;
```

- Each procedure ***F*** will be replaced by

```
wait(mutex);
    …
        body of F;

    …
if (next_count > 0)
   signal(next)
else
   signal(mutex);
```

- Mutual exclusion within a monitor is ensured



Monitor Implementation – Condition Variables

- For each condition variable ***x***, we have:

```
semaphore x_sem; // (initially = 0)
int x_count = 0;
```

- The operation x.wait can be implemented as:

```
x-count++;
if (next_count > 0)
      signal(next);
else
      signal(mutex);
wait(x_sem);
x-count--;
```



Monitor Implementation (Cont.)

- The operation x.signal can be implemented as:

```
if (x-count > 0) {
    next_count++;
    signal(x_sem);
    wait(next);
    next_count--;
}
```



Resuming Processes within a Monitor

- If several processes queued on condition x, and x.signal() executed, which should be resumed?
- FCFS frequently not adequate
- **conditional-wait** construct of the form x.wait(c)
  - Where c is **priority number**
  - Process with lowest number (highest priority) is scheduled next



A Monitor to Allocate Single Resource

```
monitor ResourceAllocator
{
    boolean busy;
    condition x;
    void acquire(int time) {
                        if (busy)
                              x.wait(time);
                        busy = TRUE;
    }
    void release() {
                        busy = FALSE;
                        x.signal();
    }
initialization code() {
     busy = FALSE;
    }
}
```



Thank you